

AUTHENTIFICATION & GESTION DES IDENTITÉS

Keycloak × GeoNature

Mettre en place un SSO OpenID Connect et brancher GeoNature dessus avec le provider [KeycloakOrganismProvider](#)

01

Pourquoi Keycloak

02

Architecture cible

03

Configurer Keycloak

04

Contrôle d'accès

05

Configurer GeoNature

06

Thème et mise en production

Déroulé

01 — Pourquoi Keycloak

Le constat UsersHub, ce qu'est un IAM, OIDC en deux minutes, le comparatif

02 — Architecture cible

Trois clients OIDC, arborescence de groupes, mapping claims → tables

03 — Configurer Keycloak

Realm, groupes et attributs, client scopes, mapper **groups**, clients

04 — Contrôle d'accès

Rôle client **access** et script authenticator Client Access Guard

05 — Configurer GeoNature

Bloc TOML, puis le code du provider custom, méthode par méthode

06 — Thème et prod

Page de login aux couleurs de l'instance, checklist, dépannage, sécurité

01

Pourquoi Keycloak

Le constat UsersHub · ce qu'est un serveur d'identités · OpenID Connect en deux minutes · ce que l'on gagne, et ce qui ne change pas

Le point de départ : UsersHub

Ce qu'UsersHub fait très bien

- Le **modèle de données métier** : **t_roles**, **bib_organismes**, **cor_roles**, profils par application
- Une base **utilisateurs** partagée par tout l'écosystème PnX (GeoNature, TaxHub, UsersHub)
- Une IHM d'administration des comptes, groupes et organismes
- L'authentification locale via **pypnusershub**, avec cookie et JWT

Ce qu'il ne fait pas

- Pas de **SSO** au-delà de l'écosystème PnX : chaque outil tiers redemande un mot de passe
- Pas de **MFA**, pas de WebAuthn, politique de mot de passe minimale
- Pas de **fédération** LDAP / Active Directory / FranceConnect
- Pas de standard **OAuth2** exploitable proprement par une application mobile
- Pas de délégation d'administration ni de journal d'audit des sessions

Keycloak : un serveur d'identités, pas une base d'utilisateurs de plus

Projet open source porté par Red Hat (CNCF), écrit en Java / Quarkus. Il centralise l'authentification et expose des identités aux applications via des protocoles standards. Les applications ne voient jamais le mot de passe.

Protocoles

OpenID Connect, OAuth 2.0, SAML 2.0.
Découverte automatique via **.well-known**

SSO

Une session pour toutes les applications du realm, avec déconnexion globale

Fédération

LDAP, Active Directory, ou un autre fournisseur OIDC / SAML en amont

Sécurité

MFA TOTP et WebAuthn, brute force detection, politique de mots de passe

Self-service

Mot de passe oublié, vérification d'email, console de compte, inscription

Extensibilité

Thèmes, flows d'authentification, providers Java ou JavaScript, API admin REST

Le vocabulaire, une bonne fois

Objet Keycloak	Ce que c'est	Dans notre installation
Realm	Un espace d'identités étanche : ses utilisateurs, ses clés, ses politiques	Un realm par système d'information
Client	Une application qui délègue son authentification au realm	geonature-local, geonature-sync, le client mobile
Client scope	Un paquet de mappers, donc de claims, attaché à un client	openid, profile, email, roles
Mapper	Une règle qui écrit un claim dans le token ou dans userinfo	Le mapper Group Membership → claim groups
Group	Un ensemble d'utilisateurs, hiérarchique, porteur d'attributs	/organismes/..., /applications/..., /geonature/...
Role	Une permission nommée, de realm ou de client	Le rôle client access
Flow	La suite d'étapes exécutées pendant une connexion	browser-rnf, avec Client Access Guard

OpenID Connect : Authorization Code Flow + PKCE

01 – Redirection

L'application envoie l'utilisateur chez Keycloak

```
GET /protocol/openid-connect/auth
?response_type=code
&code_challenge_method=S256
```

02 – Authentification

Keycloak vérifie l'identité, l'application ne voit rien

Mot de passe, MFA, puis les étapes du browser flow

03 – Code d'autorisation

Retour sur la redirect URI avec un code à usage unique

```
/api/auth/authorize/keycloak
?code=...&state=...
```

04 – Échange

Le backend échange le code contre des tokens

```
POST /token
code + client_secret
+ code_verifier
```

05 – Profil

Le backend lit le profil sur l'endpoint userinfo

```
GET /userinfo
→ sub, email, groups...
```

06 – Session locale

GeoNature crée sa propre session et son JWT

C'est ici qu'intervient le provider custom

Ce que l'on gagne concrètement

Besoin	Authentification UsersHub	Authentification Keycloak
Protocole	Propriétaire, cookie et JWT maison	OIDC, OAuth 2.0, SAML
SSO entre applications	Limité à l'écosystème PnX	Toutes les applications du realm
Double authentification	Absente	TOTP, WebAuthn, par flow
Annuaire d'entreprise	Absent	LDAP, AD, OIDC ou SAML amont
Application mobile	Login et mot de passe transmis	Code flow public avec PKCE
Cycle de vie des comptes	Manuel dans UsersHub	Self-service, inscription, reset
Sessions et audit	Peu outillé	Sessions, événements, révocation
Droits métier GeoNature	Modèle utilisateurs (inchangé)	Modèle utilisateurs (inchangé)

Keycloak répond à « **qui es-tu ?** ».
GeoNature reste seul à répondre à « **qu'as-tu le droit de faire ?** »

Keycloak porte

L'identité, le mot de passe, la session
SSO, le droit d'entrer dans l'application

Keycloak transporte

Le contexte métier : organisme,
réserve, appartenance aux groupes

GeoNature décide

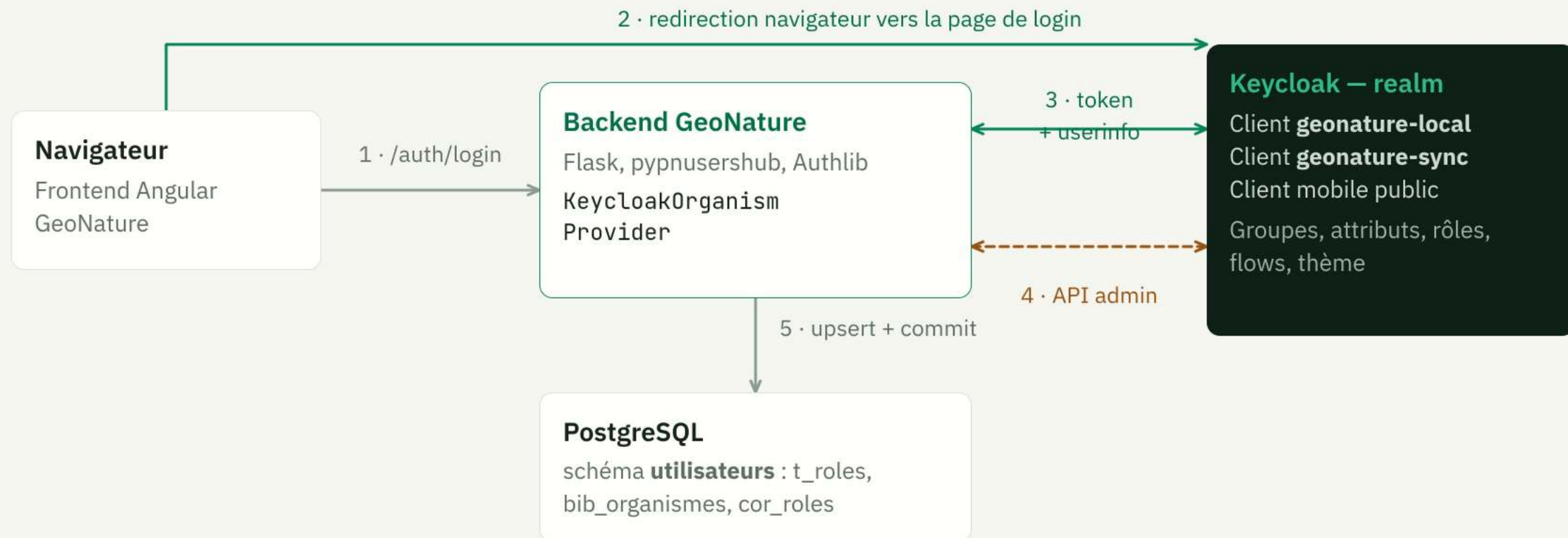
Le CRUVED, les profils par application,
la portée des données

02

L'architecture cible

Qui parle à qui · trois clients Keycloak · l'arborescence des groupes · du claim OIDC à la colonne PostgreSQL

Qui parle à qui



Trois clients Keycloak, trois usages

geonature-local

Connexion des utilisateurs web

Client **confidential**. Standard flow activé, redirect vers le backend, secret stocké côté serveur. C'est lui qui porte le rôle **access**.

geonature-sync

Lecture technique des groupes

Client **service account**, sans flow navigateur. Sert uniquement à appeler l'API admin pour lire les attributs des groupes organisme.

occtax-mobile

Application mobile

Client **public**, PKCE obligatoire, redirect sur un schéma d'URL natif. Aucun secret embarqué dans l'APK.

Pourquoi séparer **geonature-sync** ? Parce que lire l'API d'administration demande des droits que l'on ne veut jamais donner au client qui authentifie les utilisateurs. Un secret compromis côté web ne donne alors aucun accès à l'annuaire.

Chaque décision a un seul endroit où elle se prend

Question	Où c'est décidé	Mécanisme
Qui est l'utilisateur ?	Keycloak	Realm, utilisateurs, flow d'authentification
Peut-il entrer dans GeoNature ?	Keycloak	Rôle client access + script Client Access Guard
De quel organisme relève-t-il ?	Keycloak, appliqué par GeoNature	Groupe /organismes/... et ses attributs
Dans quels groupes GeoNature ?	Configuration GeoNature	group_mapping dans le TOML
Que peut-il faire sur la donnée ?	GeoNature seul	CRUVED, profils, permissions

Une arborescence de groupes, quatre intentions

```
/applications
  geonature-local
  occtax-mobile
/organismes
  rn-test
  pnr-exemple
/reserves
  42
/geonature
  test
  Grp_admin
```

/applications/... — droit d'entrée. Le groupe porte le rôle client **access**.

/organismes/... — organisme de rattachement. Porte les attributs lus par le provider.

/reserves/... — contexte métier réserve, réservé aux usages à venir.

/geonature/... — clés de **group_mapping** vers les groupes GeoNature.

LE POINT QUI FAIT PERDRE UNE DEMI-JOURNÉE

GeoNature lit les groupes dans **userinfo**, pas dans le JWT

Lu par GeoNature

L'endpoint **/userinfo**, appelé par Authlib après l'échange du code

Non lu

L'access token et l'ID token : utiles pour déboguer, ignorés par le code

Non lu non plus

realm_access.roles et **resource_access** : les rôles ne servent pas côté GeoNature

Conséquence pratique : sur le mapper Group Membership, **Add to userinfo** doit être activé. Sans lui, le token contient bien les groupes, l'organisme n'est jamais résolu, et rien n'apparaît dans les logs.

Du claim OIDC à la colonne PostgreSQL

Claim userinfo	Destination GeoNature	Remarque
sub	utilisateurs.t_roles.uuid_role	UUID stable de l'utilisateur Keycloak
preferred_username	t_roles.identifiant	Réglé par IDENTIFIER_FIELD
email	t_roles.email	Sert aussi de clé de réconciliation possible
given_name / family_name	t_roles.prenom_role / nom_role	Champs obligatoires côté Keycloak
groups	t_roles.id_organisme, cor_roles	Via le préfixe organisme et group_mapping
Attributs de groupe	utilisateurs.bib_organismes	Lus par l'API admin, pas par le token

03

Configurer Keycloak, écran par écran

Prérequis serveur · realm · groupes et attributs · client scopes · le mapper **groups** · les clients web et service · les utilisateurs

Préparer le serveur Keycloak

Activer la feature scripts

Dans `/opt/keycloak/conf/keycloak.conf`, nécessaire au script authenticator de la section 4.

```
features=preview,scripts
```

Démarrage en production

```
sudo /opt/keycloak/bin/kc.sh start --optimized
```

Rebuild après tout ajout de provider

```
sudo systemctl stop keycloak
sudo /opt/keycloak/bin/kc.sh build
sudo systemctl start keycloak
```

Sans **build**, le JAR n'est pas chargé et Keycloak loggue seulement :

A provider JAR was updated since the last build,
please rebuild

En développement, après chaque modification CSS du thème :

```
sudo rm -rf /opt/keycloak/data/tmp/kc-gzip-cache
```

Les réglages de realm qui comptent

Login

User registration **OFF** en production · Forgot password **ON** · Login with email **ON** · Edit username **OFF** pour garder un **preferred_username** stable

Localization

Internationalization **ON**, locale par défaut **fr**

Tokens et sessions

Signature **RS256** · Access token 5 à 15 min · SSO idle 30 min, max 10 h · Revoke refresh token **ON**

Security defenses

Brute force detection **ON**, en-têtes de sécurité selon la politique d'infrastructure

À noter pour la suite

L'URL du realm est l'**ISSUER** que GeoNature mettra dans son TOML :

<https://keycloak.example.org/realms/si-rnf>

Realm settings → General → OpenID Endpoint Configuration donne tous les endpoints dérivés.

Créer les groupes et porter l'organisme en attributs

Groups → **Create group**, les parents d'abord. Puis, sur chaque groupe organisme, onglet **Attributes** :

Clé exacte	Exemple de valeur	Usage dans GeoNature
id_organisme	45	Clé primaire reprise telle quelle si l'organisme n'existe pas encore
uuid_organisme	a1b2c3d4-e5f6-7890-abcd-ef1234567890	Identifiant pivot entre instances
nom_organisme	Réserve naturelle test	Libellé ; à défaut le nom du groupe est utilisé

Dans Keycloak, un attribut de groupe est toujours une **liste de chaînes**. Le provider lit `attributes["id_organisme"][0]` et convertit lui-même en entier.

Ces attributs ne sont **jamais** exposés dans le token de l'utilisateur. Seule l'API admin les rend lisibles — d'où le client **geonature-sync**.

D'où viennent les claims

Keycloak n'écrit pas les claims au hasard : il assemble le token à partir des **client scopes** attachés au client, et chaque scope contient des **mappers**. Un claim manquant, c'est presque toujours un scope non assigné ou un mapper absent.

À mettre en Default sur le client

Scope	Ce qu'il apporte
openid	sub, iss, aud, exp
profile	preferred_username, given_name, family_name
email	email, email_verified
roles	rôles, et notre mapper groups
web-origins	en-têtes CORS

Le piège à connaître

Un scope en **Optional** n'est appliqué que si le client le demande explicitement. GeoNature demande **openid email profile** et rien d'autre : tout scope utile doit donc être en **Default**.

```
Scopes demandés, codés en dur dans OpenIDProvider.configure()
:  
"scope": "openid email profile"
```

Group Membership : le paramétrage le plus important

Client scopes → roles → Mappers → Add mapper → By configuration → Group Membership

Champ	Valeur	Pourquoi
Name	groups	Libellé interne du mapper
Token Claim Name	groups	Doit correspondre à group_claim_name côté GeoNature
Full group path	ON	Donne /organismes/rn-test et non rn-test : indispensable au préfixe et au mapping
Add to userinfo	ON	Obligatoire : c'est la seule source lue par GeoNature
Add to ID token	ON	Recommandé, pour déboguer
Add to access token	ON	Recommandé, pour déboguer

Ce mapper natif n'offre **aucun filtre par préfixe** : tous les groupes de l'utilisateur partent dans le claim. C'est pour cela que l'arborescence sépare clairement **/organismes**, **/applications** et **/geonature**.

Vérifier avant d'aller plus loin

```
{
  "sub": "f47ac10b-58cc-4372-a567-...",
  "preferred_username": "jdupont",
  "given_name": "Jean",
  "family_name": "Dupont",
  "email": "jean.dupont@example.org",
  "groups": [
    "/applications/geonature-local",
    "/organismes/rn-test",
    "/geonature/test/Grp_admin"
  ]
}
```

```
ISSUER=https://kc.example.org/realms/si-rnf
```

```
curl -s -H "Authorization: Bearer $TOKEN" \
  "$ISSUER/protocol/openid-connect/userinfo" \
  | jq .
```

Les quatre points à cocher

- **groups** présent dans **userinfo**, pas seulement dans le JWT
- Valeurs en chemin complet, commençant par /
- Tous les groupes attendus sont listés
- **preferred_username**, **email**, **given_name**, **family_name** renseignés

Le client geonature-local

Capability config

Paramètre	Valeur
Client authentication	ON (confidential)
Standard flow	ON
Direct access grants	OFF
Implicit flow	OFF
Service accounts roles	OFF

Les URLs, à l'exact

Champ	Valeur
Valid redirect URIs	http://localhost:8000/api/auth/autho rize/keycloak
Web origins	http://localhost:4200
Post logout redirect	http://localhost:4200/*
Root / Home URL	http://localhost:4200

La redirect URI se déduit mécaniquement de la configuration GeoNature : **{API_ENDPOINT}/auth/authorize/{id_provider}**. Pas de joker approximatif — une URI trop large est une faille, une URI fausse donne **Invalid parameter: redirect_uri**.

Onglets Advanced, Credentials et Roles

Advanced

Signature des tokens **RS256**

PKCE Code Challenge Method **S256**

Always use PKCE : **OFF** pour un client confidential

GeoNature envoie quand même le challenge PKCE : le paramètre **CODE_CHALLENGE_METHOD** du TOML doit valoir **S256**.

Credentials

Client Authenticator : **Client Id and Secret**

Le secret se recopie dans **CLIENT_SECRET** côté GeoNature.

Tout secret utilisé pendant une phase de mise au point doit être régénéré avant la mise en service.

Roles

Créer un rôle client nommé **access**.

Il ne sert à rien tant que le script authenticator de la section suivante n'est pas en place — mais il se crée ici, une fois pour toutes.

Requête réellement émise par GeoNature au début du flow :

```
GET {ISSUER}/protocol/openid-connect/auth?response_type=code&client_id=geonature-local
  &redirect_uri=.../api/auth/authorize/keycloak&scope=openid+email+profile
  &state=...&code_challenge=...&code_challenge_method=S256
```

Le client geonature-sync et son service account

Paramètre	Valeur
Client authentication	ON
Service accounts roles	ON
Standard flow	OFF
Direct access grants	OFF
Redirect URIs / Web origins	vides

Service account roles → filtrer sur **realm-management** → assigner **query-groups**. C'est le droit minimum, et il suffit.

Ce que GeoNature appelle avec ce client

```
# 1 · jeton technique
POST {ISSUER}/protocol/openid-connect/token
  grant_type=client_credentials

# 2 · lecture du groupe
GET {HOST}/admin/realms/{REALM}
  /group-by-path/organismes/rn-test
```

Sans **query-groups**, la lecture échoue **silencieusement** côté utilisateur : la connexion réussit, l'organisme n'est pas résolu, et seul un **warning** apparaît dans les logs du backend.

Les champs obligatoires côté Keycloak

Champ Keycloak	Colonne GeoNature
Username	t_roles.identifiant
Email	t_roles.email
First name	t_roles.prenom_role
Last name	t_roles.nom_role
Enabled	condition de connexion

Un prénom ou un nom vide fait **échouer** la création du rôle GeoNature : le provider lit **user_info["given_name"]** sans valeur de repli.

Onglet **Groups** → **Join Group** : rattacher l'utilisateur à au moins **/applications/geonature-local** et à son groupe **/organismes/...**

Onglet **Credentials** : mot de passe, **Temporary = OFF** pour un compte de test.

RÉCAPITULATIF

Les huit paramétrages sans lesquels rien ne marche

#	Où	Quoi
1	Client scopes → roles → Mappers	Group Membership, claim groups, full path ON, userinfo ON
2	Client → Client scopes	openid, profile, email, roles en Default
3	Client geonature-local	Standard flow ON, redirect URI exacte, secret récupéré
4	Client geonature-sync	Service account ON, rôle query-groups
5	Groups	Arborescence /organismes, /applications, /geonature
6	Group → Attributes	id_organisme, uuid_organisme, nom_organisme
7	Group → Role mapping	/applications/<client> → rôle client access
8	Users	Username, email, prénom, nom, appartenance aux groupes

04

Le contrôle d'accès applicatif

Qui a le droit d'entrer ? Rôle client **access** · attribution par groupe · script authenticator **Client Access Guard** · browser flow dédié

Le realm authentifie tout le monde. GeoNature ne concerne que certains.

Sans garde-fou

N'importe quel compte du realm se connecte à GeoNature. Le provider crée alors un **t_roles**, l'associe à un organisme et au groupe de réconciliation par défaut.

On se retrouve avec des comptes GeoNature créés par accident, à nettoyer à la main.

Avec le rôle access

Keycloak refuse la connexion **avant** de délivrer le code. GeoNature n'est jamais appelé, aucune ligne n'est créée en base.

Le même mécanisme protège toutes les applications du realm, avec un rôle **access** par client.

Keycloak n'a pas de case « autoriser tout le monde sur ce client ». Trois approches : un groupe parent **/applications** mappé vers **access**, un rôle de realm composite, ou un script assoupli qui ignore certains clients.

Trois clics pour câbler le droit d'entrée

01

Créer le rôle

Clients → **geonature-local** → Roles →
Create role
`access`

02

Créer le groupe

Groups → Create group
`/applications`
`/geonature-local`

03

Lier les deux

Groups → le groupe → Role mapping →
Assign role → filtrer par client
`geonature-local:access`

Rien de tout cela n'est **appliqué** tant que le script de la slide suivante n'est pas branché dans le browser flow. Le rôle existe, il est attribué, mais aucune étape de connexion ne le vérifie.

Un script authenticator, livré comme un JAR

Arborescence

```
kc-script-auth/  
├─ META-INF/  
│   └─ keycloak-scripts.json  
└─ client-access-guard.js
```

Packaging et déploiement

```
jar cf client-access-guard.jar META-INF \  
    client-access-guard.js  
sudo cp client-access-guard.jar \  
    /opt/keycloak/providers/  
sudo /opt/keycloak/bin/kc.sh build  
sudo systemctl restart keycloak
```

Le manifeste META-INF/keycloak-scripts.json

```
{  
  "authenticators": [  
    {  
      "name": "Client Access Guard",  
      "fileName": "client-access-guard.js",  
      "description": "Require client role"  
    }  
  ]  
}
```

Le champ **name** est celui qui apparaîtra dans **Authentication** → **Flows** → **Add execution**. S'il n'y apparaît pas, c'est que la feature **scripts** est absente ou que le **build** n'a pas été rejoué.

client-access-guard.js

```
var AuthenticationFlowError = Java.type("org.keycloak.authentication.AuthenticationFlowError");
var REQUIRED_ROLE = "access";
var EXCLUDED_CLIENTS = { "account-console": true, "security-admin-console": true, "admin-cli": true };

function authenticate(context) {
    var user = context.getUser();
    if (user == null) { context.attempted(); return; } // étape précédente non jouée

    var client = context.getAuthenticationSession().getClient();
    if (client == null || EXCLUDED_CLIENTS[String(client.getClientId())]) { context.success(); return; }

    var role = client.getRole(REQUIRED_ROLE);
    if (role != null && user.hasRole(role)) { context.success(); return; }

    context.getEvent().error("client_role_missing"); // journalisé côté Keycloak
    var challenge = context.form().setError("invalidUserMessage").createLoginUsernamePassword();
    context.failureChallenge(AuthenticationFlowError.INVALID_USER, challenge);
}
```

user.hasRole() interroge le moteur de Keycloak, pas le JWT : les rôles hérités des groupes comptent.

Le fichier doit n'exposer que des **fonctions** ; un **return** au niveau global donne **ScriptCompilationException**.

Message volontairement générique : ne jamais révéler que le compte existe mais n'a pas le rôle.

Brancher le script dans le browser flow

Authentication → Flows

1. Dupliquer le flow **browser** en **browser-rnf**
2. Dans le sous-flow **forms**, ajouter les exécutions dans cet ordre
3. Onglet **Bindings** → Browser Flow = **browser-rnf**

Exécution	Requirement
Username Password Form	REQUIRED
Client Access Guard	REQUIRED
Browser - Conditional 2FA	CONDITIONAL

Le message affiché à l'utilisateur refusé

Dans le thème de login, fichier **messages/messages_fr.properties** :

```
invalidUserMessage=Nom d'utilisateur ou  
mot de passe invalide ou accès non autorisé  
loginTitle=Se connecter au {0}
```

Un seul message pour trois situations : mauvais mot de passe, compte inconnu, accès refusé. C'est volontaire — on ne donne aucune information exploitable.

05

Configurer GeoNature, du TOML au provider

Le bloc **[AUTHENTICATION]**, le chargement du provider, puis le code de **KeycloakOrganismProvider**, méthode par méthode

config/geonature_config.toml

```
[AUTHENTICATION]
DEFAULT_RECONCILIATION_GROUP_ID = 1

# toujours conserver un accès local
[[AUTHENTICATION.PROVIDERS]]
module = "pypnusershub.auth.providers
        .default.LocalProvider"
id_provider = "local_provider"

[[AUTHENTICATION.PROVIDERS]]
module = "geonature.keycloak_provider
        .KeycloakOrganismProvider"
id_provider = "keycloak"

ISSUER = "https://kc.example.org/realms/si-rnf"
CLIENT_ID = "geonature-local"
CLIENT_SECRET = "..."
```

```
# hérité de OpenIDConnectProvider
group_claim_name = "groups"
IDENTIFIER_FIELD = "preferred_username"
RECONCILIATE_ATTR = "email"
CODE_CHALLENGE_METHOD = "S256"

# propres au provider custom
ORGANISM_GROUP_PREFIX = "/organismes/"
ORGANISM_UUID_CLAIM = "uuid_organisme"
ORGANISM_NAME_CLAIM = "organisme"
USER_UUID_CLAIM = "sub"

KEYCLOAK_ADMIN_CLIENT_ID = "geonature-sync"
KEYCLOAK_ADMIN_CLIENT_SECRET = "..."
KEYCLOAK_ADMIN_TIMEOUT = 5

[AUTHENTICATION.PROVIDERS.group_mapping]
"/geonature/test" = 1
"/geonature/test/Grp_admin" = 2
```

Ce que chaque clé pilote

Clé	Effet
ISSUER	URL du realm ; sert à découvrir les endpoints OIDC et à dériver l'URL de l'API admin
IDENTIFIER_FIELD	Claim utilisé comme identifiant GeoNature
RECONCILIATE_ATTR	Champ de rapprochement avec un compte existant, côté classe parente
group_claim_name	Nom du claim contenant les chemins de groupes
ORGANISM_GROUP_PREFIX	Préfixe qui désigne un groupe organisme parmi tous les groupes reçus
USER_UUID_CLAIM	Claim recopié dans t_roles.uuid_role ; sub par défaut
KEYCLOAK_ADMIN_*	Identifiants du service account et délai d'expiration des appels admin
group_mapping	Chemin de groupe Keycloak → id_role d'un groupe GeoNature
DEFAULT_RECONCILIATION_GROUP_ID	Groupe attribué quand aucun mapping ne s'applique

Garder **local_provider** déclaré : sans lui, le frontend redirige automatiquement vers Keycloak, et la moindre erreur de configuration produit une boucle de redirection dont on ne sort plus par l'interface.

Comment le provider est chargé au démarrage

backend/geonature/app.py

```
auth_manager.init_app(
    app,
    providers_declaration=config
        ["AUTHENTICATION"]["PROVIDERS"],
)
```

pypnusershub/auth/auth_manager.py

```
module = importlib.import_module(import_path)
class_ = getattr(module, class_name)
instance_provider = class_()
instance_provider.configure(
    configuration=provider_config)
self.add_provider(
    instance_provider.id_provider,
    instance_provider)
```

Ce que cela implique

- La clé **module** du TOML est un **chemin d'import Python** : le provider peut vivre dans GeoNature, dans un module tiers ou dans **pypnusershub**
- Aucune inscription au registre, aucun point d'entrée **setuptools** : il suffit que la classe soit importable
- **id_provider** devient le segment d'URL des routes **/auth/login/<id>** et **/auth/authorize/<id>**
- Plusieurs providers OIDC peuvent coexister, chacun avec son propre bloc de configuration

Écrire un provider custom revient donc à sous-classer, surcharger **configure()** et **authorize()**, et changer une ligne de TOML.

KeycloakOrganismProvider, en une page

Authentication



OpenIDProvider
authorize, revoke, configure



OpenIDConnectProvider
+ end_session_endpoint



KeycloakOrganismProvider
+ organismes, uuid_role, API admin

Ce qu'il ajoute

- Résout l'**organisme** depuis le groupe **/organismes/...** et ses attributs
- Crée ou met à jour **bib_organismes**, puis renseigne **t_roles.id_organisme**
- Recopie le claim **sub** dans **t_roles.uuid_role**, après validation du format UUID
- Force la réconciliation sur **identifiant** plutôt que sur l'email

Ce qu'il ne fait pas

- Aucun contrôle d'accès applicatif : c'est le rôle de Keycloak
- Aucune synchronisation miroir des groupes à chaque reconnexion

configure() — valider la configuration au démarrage

```
class KeycloakOrganismProvider(OpenIDConnectProvider):
    def configure(self, configuration):
        super().configure(configuration) # ISSUER, CLIENT_ID/SECRET, scopes, PKCE, Authlib
        class KeycloakOrganismConfiguration(ProviderConfigurationSchema):
            ORGANISM_GROUP_PREFIX = fields.String(load_default="/organismes/")
            ORGANISM_UUID_CLAIM    = fields.String(load_default="uuid_organisme")
            USER_UUID_CLAIM       = fields.String(load_default="sub")
            KEYCLOAK_ADMIN_CLIENT_ID = fields.String(load_default=None, allow_none=True)
            KEYCLOAK_ADMIN_CLIENT_SECRET = fields.String(load_default=None, allow_none=True)
            KEYCLOAK_ADMIN_TIMEOUT   = fields.Integer(load_default=5)
        conf = KeycloakOrganismConfiguration().load(configuration, unknown=EXCLUDE)
        self.group_prefix = conf["ORGANISM_GROUP_PREFIX"] # ... et les autres clés
        self.keycloak_issuer = configuration.get("ISSUER") # réutilisé pour l'API admin
```

Un schéma **marshmallow** : une clé mal orthographiée dans le TOML lève une **ValidationError** au démarrage, pas à la première connexion.

Tous les paramètres ont une valeur par défaut : un TOML minimal suffit si l'on respecte les conventions de nommage.

authorize() — le callback, de bout en bout

```
def authorize(self):
    oauth_provider = getattr(oauth, self.id_provider)
    token = oauth_provider.authorize_access_token() # 1 · code → tokens → userinfo
    session["openid_token_resp"] = token # conservé pour la déconnexion OIDC
    user_info = token["userinfo"]
    source_groups = user_info.get(self.group_claim_name, []) # 2 · les groupes
    organism = self._resolve_organism(user_info, source_groups) # 3 · l'organisme
    new_user = {"identifiant": user_info[self.identifiant_field],
                "email": user_info["email"],
                "prenom_role": user_info["given_name"],
                "nom_role": user_info["family_name"], "active": True}
    if keycloak_user_uuid: new_user["uuid_role"] = keycloak_user_uuid # 4 · sub validé
    if organism: new_user["id_organisme"] = organism.id_organisme
    user = self.insert_or_update_role( # 5 · upsert + group_mapping
        new_user, source_groups=source_groups, reconcile_attr="identifiant")
    db.session.commit()
    return user
```

`_resolve_organism()` — la cascade de résolution

```
org_uuid = user_info.get(self.organism_uuid_claim) # si un claim direct existe
org_name = user_info.get(self.organism_name_claim)
if not org_uuid or not org_name: # sinon, on interroge l'API admin
    path = self._extract_first_group_organism_path(source_groups) # /organismes/...
    attrs = (self._get_group_attributes_from_keycloak(path) or {}).get("attributes", {})
    org_id = int(attrs["id_organisme"][0]) # attribut = liste de chaînes
    org_uuid = org_uuid or attrs["uuid_organisme"][0]

organism = lookup(id_organisme) or lookup(uuid_organisme) or lookup(nom_organisme)
if not organism:
    organism = models.Organisme(nom_organisme=org_name); db.session.add(organism)
```

1 · id_organisme

reprise d'un identifiant existant

2 · uuid_organisme

pivot entre instances

3 · nom_organisme

dernier recours

4 · création

sinon, nouvel organisme

Le canal service account, avec cache de jeton

```
def _get_kc_admin_token(self):
    if self._kc_admin_token and time.time() < self._kc_admin_token_exp:
        return self._kc_admin_token # jeton encore valide, aucun appel réseau
    resp = requests.post(f"{self.keycloak_issuer}/protocol/openid-connect/token",
        data={"grant_type": "client_credentials",
            "client_id": self.keycloak_admin_client_id,
            "client_secret": self.keycloak_admin_client_secret},
        timeout=self.keycloak_admin_timeout)
    payload = resp.json()
    self._kc_admin_token = payload.get("access_token")
    self._kc_admin_token_exp = time.time() + max(payload.get("expires_in", 60) - 10, 10)
    return self._kc_admin_token
```

Appel nominal — **GET /admin/realms/{realm}/group-by-path/organismes/rn-test**. L'URL de base est dérivée de l'ISSUER en remplaçant **/realms/** par **/admin/realms/**.

Repli — si l'endpoint échoue, **GET /groups?search=<feuille>&briefRepresentation=false**, puis parcours récursif des sous-groupes jusqu'à retrouver le **path** exact.

Des groupes Keycloak vers les groupes GeoNature

```
[AUTHENTICATION.PROVIDERS.group_mapping]
"/geonature/test" = 1
"/geonature/test/Grp_admin" = 2
```

Clé : le chemin exact tel qu'il apparaît dans **userinfo.groups**. Valeur : l'**id_role** d'un groupe GeoNature, c'est-à-dire une ligne de **t_roles** avec **groupe = true**.

```
SELECT r.identifiant, g.id_role AS groupe
FROM utilisateurs.t_roles r
LEFT JOIN utilisateurs.cor_roles cr
  ON cr.id_role_utilisateur = r.id_role
LEFT JOIN utilisateurs.t_roles g
  ON g.id_role = cr.id_role_groupe
WHERE r.identifiant = 'mon_user_test';
```

Le comportement, sans surprise

Situation	Résultat
Nouvel utilisateur, mapping trouvé	Groupes affectés à la création
Nouvel utilisateur, aucun mapping	DEFAULT_RECONCILIATION_GROUP_ID
Utilisateur déjà existant	Groupes non resynchronisés

Conséquence : retirer un utilisateur d'un groupe Keycloak ne lui retire pas son groupe GeoNature. La synchronisation « miroir » à chaque connexion reste à implémenter dans **insert_or_update_role**.

Pour tester un mapping, supprimer l'utilisateur GeoNature puis se reconnecter : la logique ne s'applique qu'à la création.

06

Le thème et la mise en production

Page de connexion aux couleurs de l'instance · checklist de déploiement · vérifications SQL ·
dépannage · sécurité · perspectives

Une page de login aux couleurs de l'instance

```
/opt/keycloak/themes/rnf/  
└─ login/  
    └─ theme.properties  
    └─ resources/  
        └─ css/styles.css  
        └─ img/logo.png  
    └─ messages/  
        └─ messages_fr.properties  
    └─ footer.ftl
```

```
theme.properties  
parent=keycloak.v2  
styles=css/styles.css
```

Activation

Realm settings → **Themes** → **Login theme** = **rnf**. Le thème hérite de **keycloak.v2** : on ne redéfinit que ce que l'on veut changer.

La clé **styles remplace** la liste du parent, elle ne s'y ajoute pas. Y référencer **css/login.css** sans fournir le fichier provoque un 404.

Ce que le thème permet aussi

- Le message générique d'accès refusé, via **messages_fr.properties**
- Un lien d'inscription externe, via une macro **footer.ftl**
- Le lien « mot de passe oublié », activé dans Realm settings → Login

styles.css : les sélecteurs qui fonctionnent

```
/* fond institutionnel */
body.kcBodyClass, .pf-v5-c-login {
  background: #00885b !important;
}

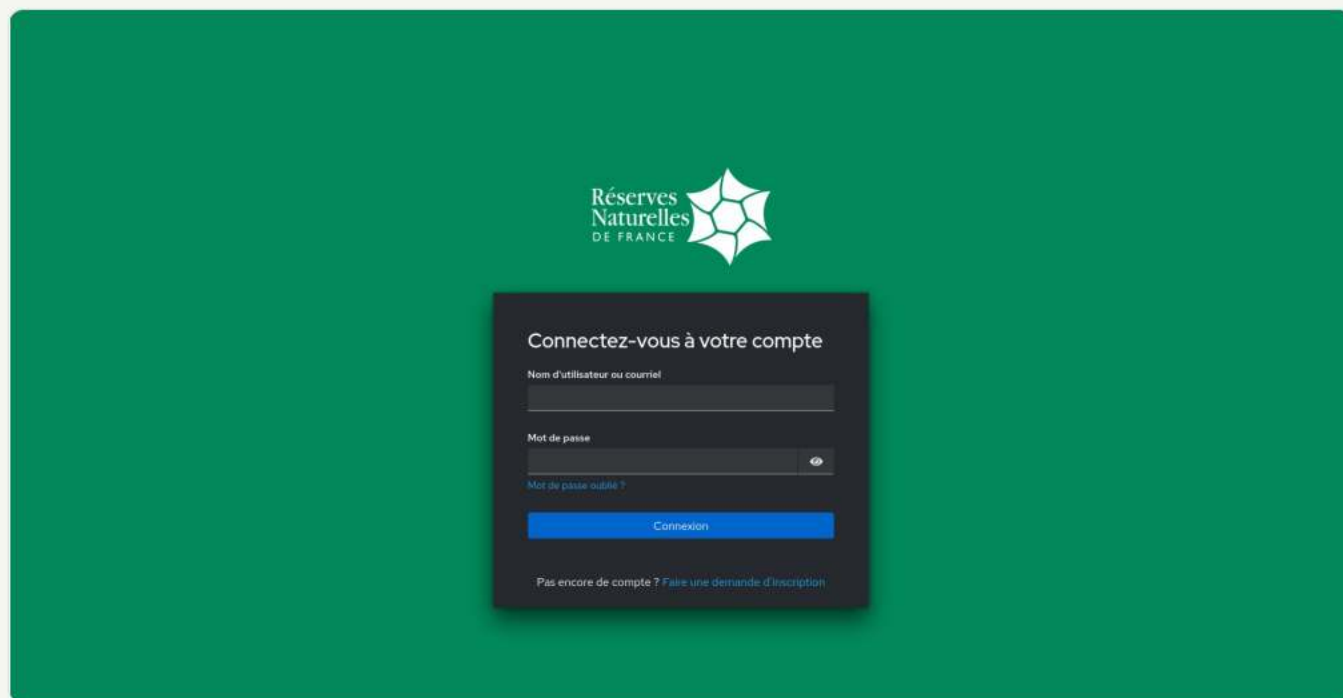
/* logo à la place du titre */
#kc-header-wrapper.pf-v5-c-brand {
  display: block;
  width: 340px; height: 120px;
  margin: 0 auto;
  background: url("../img/logo.png")
    no-repeat center center;
  background-size: contain;
}
```

Les pièges rencontrés

Symptôme	Cause et remède
CSS modifié, rien ne change	Cache gzip : purger kc-gzip-cache, redémarrer, Ctrl+F5
Logo invisible	Mauvais sélecteur : viser #kc-header-wrapper
content: url(...) sans effet	Utiliser background: url(...)
404 sur css/login.css	Fichier déclaré mais absent du thème
Logo aligné à droite	Forcer une grille CSS sur .pf-v5-c-login__container

Les classes **pf-v5-*** viennent de PatternFly 5, le socle graphique de **keycloak.v2** : elles changent d'une version majeure de Keycloak à l'autre, à revalider lors des montées de version.

Le résultat : la page vue par l'utilisateur



Le logo remplace le titre

Sélecteur **#kc-header-wrapper.pf-v5-c-brand**, image posée en **background**, texte d'origine masqué

Le fond institutionnel

body.kcBodyClass et **.pf-v5-c-login**, en **!important** pour passer devant le thème parent

Le lien d'inscription

Ajouté par la macro **footer.ftl** du thème, il pointe vers le formulaire de demande de compte

« Mot de passe oublié ? »

Pas de CSS ici : **Realm settings** → **Login** → **Forgot password**, plus la configuration SMTP

Déployer sur un nouvel environnement, dans l'ordre

Côté Keycloak

1. Créer le realm
2. Activer **features=scripts**, puis **kc.sh build**
3. Créer les clients web, service et mobile
4. Créer le rôle **access** et les groupes **/applications/...**
5. Configurer le mapper **groups**
6. Créer les groupes **/organismes/...** et leurs attributs
7. Déployer le JAR et basculer le browser flow
8. Déployer le thème de login

Côté GeoNature

1. Sauvegarder la base de données
2. Renseigner le bloc **[AUTHENTICATION]** du TOML
3. Déclarer le **group_mapping** et les groupes cibles
4. Redémarrer le backend
5. Vérifier **GET /api/auth/providers**
6. Tester un utilisateur autorisé
7. Tester un utilisateur **non** autorisé
8. Contrôler **uuid_role** et **id_organisme** en base

Les requêtes de contrôle après la première connexion

```
-- l'utilisateur
SELECT id_role, identifiant, uuid_role,
       id_organisme, active
FROM utilisateurs.t_roles
WHERE identifiant = 'login_test';
```

```
-- l'organisme résolu
SELECT id_organisme, uuid_organisme,
       nom_organisme
FROM utilisateurs.bib_organismes
WHERE id_organisme = 45;
```

```
-- droits sur l'application
SELECT * FROM utilisateurs
       .v_userslist_forall_applications
WHERE identifiant = 'login_test';
```

Si des **id_organisme** explicites sont injectés depuis Keycloak, resynchroniser la séquence, sinon la prochaine insertion locale entrera en collision :

```
SELECT setval(pg_get_serial_sequence(
    'utilisateurs.bib_organismes','id_organisme'),
    (SELECT COALESCE(MAX(id_organisme),1)
     FROM utilisateurs.bib_organismes), true);
```

Dépannage : les dix symptômes déjà rencontrés

Symptôme	Cause	Correction
groups absent de userinfo	Add to userinfo désactivé	Corriger le mapper
groups sans le / initial	Full group path désactivé	Activer Full group path
Invalid parameter: redirect_uri	URI non identique	Aligner client et API_ENDPOINT
MismatchingStateError, boucle	Cookies perdus, deux hostnames	Un seul hostname, vider les cookies
group path does not exist	Service account sans droits	Ajouter query-groups
Organisme non créé	Aucun groupe /organismes/	Rattacher l'utilisateur au groupe
uuid_role vide	Claim sub absent ou invalide	Vérifier USER_UUID_CLAIM
Groupes GeoNature non affectés	Utilisateur déjà existant	Recréer le compte, ou affecter à la main
Client Access Guard introuvable	Feature scripts, ou pas de build	kc.sh build puis redémarrage
Invalid return statement	return global dans le script	Tout mettre dans des fonctions

Les règles à ne pas négocier

Secrets — rotation de tous les **CLIENT_SECRET** utilisés pendant la mise au point, avant l'ouverture aux utilisateurs.

Journaux — ne jamais écrire dans les logs un token, un **code**, un **code_verifier** ou un secret.

Redirect URIs — liste blanche stricte, web comme mobile. Pas de joker sur un domaine entier.

Mobile — client public et PKCE obligatoire, jamais de secret embarqué dans l'application.

TLS de bout en bout — Keycloak, l'API et le frontend. Un code d'autorisation qui transite en clair est un compte compromis.

Moindre privilège — **geonature-sync** ne détient que **query-groups**, en lecture seule.

Messages d'erreur — un seul libellé générique à la connexion, qui ne révèle ni l'existence du compte ni l'absence de droit.

Accès de secours — conserver **local_provider** et un compte administrateur local si Keycloak tombe.

Ce qui reste devant nous

Flux mobile PKCE

Endpoint **POST**

/api/auth/mobile/keycloak, client public, redirection sur schéma natif.

Même logique de réconciliation, réponse JSON avec le JWT GeoNature.

Détaillé en section 9 du guide.

Synchronisation miroir

Appliquer **group_mapping** à chaque connexion, retraits compris, dans **insert_or_update_role**. Fait de Keycloak la source unique des groupes.

Décision d'architecture à prendre.

Fédération et MFA

Brancher un annuaire en amont, activer la double authentification pour les profils sensibles — sans toucher une ligne de GeoNature.

C'est le vrai retour sur investissement.

Documentation du dépôt — **docs/KEYCLOAK_GEONATURE.md** pour le guide exhaustif, **DEPLOYMENT_KEYCLOAK.md** pour la note de déploiement, **backend/geonature/keycloak_provider.py** pour le code du provider.

SUITES

Des applications mobiles officielles, en partenariat avec l'ONF

Aujourd'hui — du bricolage assumé

Les essais de connexion mobile menés jusqu'ici sont des **prototypes** : ils prouvent que le flux PKCE fonctionne, ils ne sont pas destinés à la production.

Seul l'endpoint backend **/api/auth/mobile/keycloak** est en place et réutilisable.

Demain — un développement commun

Avec l'**ONF**, développement de versions **officielles** d'**Occtax mobile** et de **Monitoring mobile**, capables de s'authentifier auprès d'un **fournisseur d'identité externe**.

La connexion externe devient une fonctionnalité du produit, plus un correctif local.

Pour nous — Keycloak

Client public, PKCE, redirection sur schéma natif

Pour l'ONF — Azure, Entra ID

Même protocole, autre fournisseur d'identité

Côté GeoNature — rien de spécifique

Un provider déclaré dans le TOML, comme pour le web

L'enjeu du partenariat : une seule application mobile, un seul contrat d'authentification — **OIDC, client public, PKCE** — et le fournisseur d'identité qui devient un simple paramètre de déploiement, propre à chaque établissement.

Bonus

Fédérer un annuaire LDAP

Ce qui change et ce qui ne change pas · le formulaire **User federation** champ par champ · les mappers et les pièges

Keycloak devient la façade de l'annuaire



User federation → Add LDAP provider

Champ	OpenLDAP	Active Directory
Vendor	Other	Active Directory
Connection URL	ldaps://ldap.example.org:636	ldaps://dc.example.local:636
Bind DN	cn=keycloak,ou=services,dc=...	CN=keycloak,OU=Services,DC=...
Users DN	ou=people,dc=example,dc=org	OU=Users,DC=example,DC=local
Username LDAP attribute	uid	sAMAccountName
RDN LDAP attribute	uid	cn
UUID LDAP attribute	entryUUID	objectGUID
User object classes	inetOrgPerson, organizationalPerson	person, organizationalPerson, user
Edit mode	READ_ONLY	READ_ONLY
Import users	ON	ON

Deux boutons de contrôle avant d'enregistrer : **Test connection** et **Test authentication**.

Puis l'action **Synchronize all users**, et une synchronisation périodique si l'annuaire bouge.

Un **User LDAP filter** permet de ne remonter qu'une sous-population, par exemple un groupe métier.

Mappers et pièges

Les mappers

Keycloak crée seul les mappers d'attributs : **username**, **email**, **first name**, **last name**. Ce sont exactement les champs dont GeoNature a besoin — rien à ajouter.

Pour remonter aussi les groupes de l'annuaire, ajouter un **group-ldap-mapper** :

Champ	Valeur courante
LDAP Groups DN	ou=groups,dc=example,dc=org
Group Name LDAP Attribute	cn
Group Object Classes	groupOfNames
Membership LDAP Attribute	member
Mode	READ_ONLY

Les pièges

Les groupes importés arrivent **à la racine** de l'arborescence, pas sous **/organismes/**. Garder les groupes métier côté Keycloak plutôt que d'essayer de les faire porter par l'annuaire.

En **READ_ONLY**, prénom, nom et mot de passe ne se modifient plus que dans l'annuaire. C'est voulu, mais il faut le dire aux utilisateurs.

Le **Username LDAP attribute** devient **preferred_username**, donc **t_roles.identifiant**. En changer après coup casse la réconciliation des comptes existants.

Supprimer puis recréer le provider de fédération recrée les utilisateurs côté Keycloak : nouveau **sub**, donc nouveau **uuid_role**. À éviter en production.

Questions

Les trois choses à retenir

- 1 · Keycloak authentifie, GeoNature autorise.
- 2 · Le mapper **groups** doit alimenter **userinfo**.
- 3 · Les attributs d'organisme passent par l'API admin, pas par le token.

Pour rejouer l'installation

```
docs/KEYCLOAK_GEONATURE.md  
DEPLOYMENT_KEYCLOAK.md  
backend/geonature/  
keycloak_provider.py
```